



UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

ING. CIENCIAS DE DATOS Y ANALITICA

CURSO: Programacion para la Ciencia e Ingenieria II

DOCENTE: ING. Carlos Alberto Arias López

Alumno:

**LUIS EDUARDO
CUADRA REYES**



1113-25-24908

AcadNet, Proyecto 2

Introducción

En este ensayo voy a exponer y reflexionar sobre el desarrollo de AcadNet, una plataforma educativa web que desarrollé como parte del curso de Programación para la Ciencia y la Ingeniería II. AcadNet es una plataforma que conecta estudiantes con tutores dentro de un entorno académico, permitiendo gestionar sesiones de tutoría, cargar notas y generar reportes estadísticos. El sistema tiene tres partes principales: un backend hecho con Flask, una API REST que conecta todo, y un frontend construido con Django. También maneja tres tipos de usuarios: administrador, tutor y estudiante, cada uno con sus propias funcionalidades.

Desarrollo

Para construir AcadNet usé una arquitectura cliente-servidor, donde el frontend en Django se comunica con el backend en Flask a través del protocolo HTTP, usando los métodos GET y POST para intercambiar información. Django es un framework que facilita la construcción de interfaces web gracias a su sistema de plantillas basado en el patrón MVT, mientras que Flask es un framework liviano y perfecto para construir APIs REST de forma sencilla y eficiente.

Una de las partes más importantes del proyecto fue la implementación de una API REST con distintos endpoints que permiten hacer cosas como iniciar sesión, cargar archivos XML, registrar notas y generar reportes. Para probar que todo funcionara correctamente usé Postman, una herramienta visual que permite hacer peticiones HTTP sin tener que escribir código en la terminal.

En cuanto a Programación Orientada a Objetos, implementé una matriz dispersa usando clases y objetos. Esta estructura de datos guarda únicamente los valores que existen, lo que hace más eficiente el manejo de memoria al registrar las notas de los estudiantes. Para extraer los horarios desde archivos XML usé expresiones regulares, que son patrones que permiten buscar texto específico dentro de una cadena de caracteres. También usé Graphviz para generar una representación visual de la matriz de notas, y ChartJS para mostrar reportes estadísticos con gráficas interactivas en el navegador.

Durante el desarrollo apliqué buenas prácticas de versionamiento con Git y GitHub, haciendo commits frecuentes y creando releases que marcan versiones estables del proyecto. Para mantener ordenadas las librerías del proyecto usé un entorno virtual que las aisló del resto del sistema. Para instalar herramientas como Graphviz en Mac usé Homebrew, un gestor de paquetes para la terminal, lo que también requirió configurar correctamente el PATH del sistema operativo para que los ejecutables fueran reconocidos.

Las partes más difíciles del proyecto fueron varias. En primer lugar, la conexión entre el frontend y el backend requirió entender bien cómo se comunican mediante sesiones y peticiones HTTP, ya que al principio no era claro cómo la información viajaba de un servicio a otro. El desarrollo del backend fue la parte más larga y compleja, especialmente la implementación de la matriz dispersa y el procesamiento de archivos XML.

Durante el desarrollo también enfrenté varios problemas técnicos. Por ejemplo, el entorno virtual se subió accidentalmente a GitHub al inicio del proyecto, lo que requirió eliminarlo y configurar correctamente el archivo `.gitignore` para evitar que volviera a ocurrir. También tuve dificultades con Django al momento de encontrar los templates, ya que la carpeta `templates` no estaba en la ubicación correcta y el parámetro `DIRS` en el archivo `settings.py` estaba vacío, lo que generaba errores al cargar las pantallas.

Otro problema fue con Graphviz, que no era reconocido por el sistema porque no estaba configurado en el PATH del sistema operativo, y la solución fue agregarlo manualmente mediante código en el backend. También intenté cargar fuentes externas de Google Fonts dentro de un archivo SVG, pero el navegador las bloqueaba por restricciones de seguridad, por lo que tuve que mover el logo directamente al HTML para que las fuentes funcionaran correctamente. Finalmente, comprender por qué el comando `export PATH` solo aplicaba a la terminal activa y no a todas fue otro punto de aprendizaje importante durante el proceso.

Conclusiones

Hacer AcadNet me permitió recordar cosas que tenía tiempo de no usar como CSS, HTML y el patrón MVT, y al mismo tiempo poner en práctica conceptos importantes como framework, PATH, Homebrew, endpoints y arquitectura cliente-servidor. Si tuviera más tiempo, implementaría mejoras en dos áreas principales. En cuanto a seguridad, añadiría autenticación mediante tokens JWT para proteger los endpoints de la API, implementaría hashing de contraseñas con bcrypt en lugar de almacenarlas en texto plano, y agregaría protección contra ataques de inyección XML y fuerza bruta en el login. En cuanto al rendimiento con grandes volúmenes de datos, migraría el almacenamiento de datos de memoria RAM a una base de datos relacional como PostgreSQL, implementaría paginación en las consultas para evitar sobrecargar el servidor, y optimizaría la matriz dispersa para manejar operaciones con complejidad $O(1)$ en lugar de $O(n)$. Estas mejoras harían de AcadNet una plataforma más robusta, segura y escalable. En general fue un proyecto desafiante que me ayudó a crecer como desarrollador y a entender mejor cómo funciona el desarrollo de software moderno.

LINK DE REPO GITHUB:

https://github.com/lcuadrar/PPCYL2_PSemestre_Proyecto2_1113-25-24908